

Reviewer Pack — Monomer-Dimer Entropy Lower-Bounds Tseitin Resolution Width

Entry c5c4b106a0cf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 13:54:06 UTC

Monomer-Dimer Entropy Lower-Bounds Tseitin Resolution Width

Entry ID: c5c4b106a0cf **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 13:54:06 UTC

1. Conjecture

Field A (mathematical branch): Statistical mechanics on graphs / monomer-dimer theory: Heilmann-Lieb real-rootedness of the matching polynomial $m_G(x) = \sum_k (-1)^k m_k(G) x^{n-2k}$ and the resulting per-site free energy $h_{md}(G) = (1/n) \log \sum_k m_k(G)$, a Heilmann-Lieb / Godsil / Gurvits invariant rarely deployed in proof complexity and distinct from spectral expansion. **Field B** (complexity object): Resolution refutation width $w(T(G, c) \vdash \perp)$ of Tseitin formulas on 3-regular graphs G with odd total F_2 -charge c — the Urquhart / Ben-Sasson-Wigderson / Alekhnovich-Razborov target.

Statement:

There exists an absolute constant $\alpha > 0$ such that for every 3-regular graph G on n vertices and every charge $c \in F_2^{V(G)}$ with $\sum_v c_v = 1$, the resolution refutation width satisfies $w(T(G, c) \vdash \perp) \geq \alpha \cdot n \cdot h_{md}(G)$, where $h_{md}(G) := (1/n) \cdot \log_2 \sum_{k=0}^{\lfloor n/2 \rfloor} m_k(G)$. A single 3-regular graph on at most 20 vertices with odd charge whose Tseitin formula admits a resolution refutation of width strictly below $0.05 \cdot n \cdot h_{md}(G)$ refutes the conjecture.

Rationale (proposer's reasoning):

Heilmann-Lieb log-concavity makes h_{md} a topological-combinatorial 'routing entropy' of unit F_2 -flows in the cycle space, and these routings are exactly the obstruction narrow resolution must clear to cancel an odd parity. Unlike edge expansion, h_{md} is girth-sensitive and survives moderate bottlenecks, so the conjecture predicts width bounds strictly stronger than Ben-Sasson-Wigderson on low-expansion-but-high- h_{md} graphs such as long prisms or generalized Möbius-Kantor families. The tie to extended Frege is downstream: if the bound saturates the BSW expansion bound only on Ramanujan-type graphs, the slack pinpoints where higher proof systems can compress.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1963a7fce33f3194

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 80 instances (3-regular graphs $n \in \{4, 6, 8, 10, 12, 14\}$, ≥ 5 odd charges each), compute ratio $r = w(T(G, c) \vdash \perp) / (n \cdot h_{md}(G))$. SUPPORTED iff $\min r \geq 0.05$ over all instances AND mean $r \geq 0.10$. FALSIFIED iff any single instance yields $r < 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tseitin formulas resolution width matching polynomial - monomer-dimer entropy proof complexity expander Tseitin - resolution width lower bound 3-regular graph matching Heilmann-Lieb

Top relevant hits considered: - [<http://arxiv.org/abs/2103.09609v1>] Characterizing Tseitin-formulas with short regular resolution refutations - [<http://arxiv.org/abs/astro-ph/0407622v3>] A Study of the Composition of Ultra High Energy Cosmic Rays Using the High Resolution Fly's Eye - [<http://arxiv.org/abs/2209.05839v3>] On bounded depth proofs for Tseitin formulas on the grid; revisited - [<http://arxiv.org/abs/1809.06500v3>] Range entropy: A bridge between signal complexity and self-similarity - [<http://arxiv.org/abs/2309.13632v2>] Decay of correlations in the monomer-dimer model - [<http://arxiv.org/abs/1509.03852v3>] Proof of Convergence for the Lattice Monomer-Dimer Cluster Expansion I, a Simplified Model - [<http://arxiv.org/abs/1709.08890v2>] Partial matching width and its application to lower bounds for branching programs - [<http://arxiv.org/abs/2004.11505v1>] On Graph Continued Fractions and the Heilmann-Lieb Theorem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            factor = 1 / A[i][i]
            for j in range(n):
```

```

        A[i][j] *= factor
    for k in range(m):
        if k != i:
            factor = A[k][i]
            for j in range(n):
                A[k][j] -= factor * A[i][j]
    return A

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def log2(x):
    return math.log2(x)

def h_md(G):
    n = len(G)
    m_k = [0] * (n // 2 + 1)
    for e in G:
        u, v = e
        m_k[sum(1 for _ in range(n) if G[_][u] == G[_][v])] += 1
    return sum(m_k) / n

def tseitin_formula(G, c):
    n = len(G)
    literals = {i: (f'x{i}', f'y{i}') for i in range(n)}
    clauses = []
    for e in G:
        u, v = e
        x_u, y_u = literals[u]
        x_v, y_v = literals[v]
        clauses.append([f'¬{y_u}', f'{x_v}'])
        clauses.append([f'¬{y_v}', f'{x_u}'])
        clauses.append([f'¬{x_u}', f'¬{x_v}', f'y_u'])
        clauses.append([f'¬{x_u}', f'¬{x_v}', f'y_v'])
    for i in range(n):
        x_i, y_i = literals[i]
        if c[i] == 1:
            clauses.append([f'{x_i}'])
        else:
            clauses.append([f'¬{x_i}'])
    return clauses

def resolution_width(clauses):
    n = len(clauses)
    resolvents = []
    for k in range(1, n + 1):
        new_resolvents = []
        for clause in clauses:
            if len(clause) > k:
                continue

```

```

        for other_clause in clauses:
            if len(other_clause) > k and set_clause.isdisjoint(set(other_clause)):
                resolvent = list(set_clause | set(other_clause))
                resolvent.remove('-' + resolvent[0])
                new_resolvents.append(resolvent)
        if not new_resolvents:
            return k - 1
        clauses.extend(new_resolvents)
    return n

def is_3_regular(G):
    for v in range(len(G)):
        if sum(1 for e in G if v in e) != 3:
            return False
    return True

def generate_random_odd_charges(n, num_charges):
    charges = []
    for _ in range(num_charges):
        charge = [random.choice([0, 1]) for _ in range(n)]
        if sum(charge) % 2 == 1:
            charges.append(charge)
    return charges

def generate_prism(n):
    G = [[i, (i + 1) % n] for i in range(n)] + [[n - 1, n // 2], [0, n // 2]]
    return G

def generate_möbius_kantor():
    G = [
        [0, 1], [1, 2], [2, 3], [3, 4], [4, 5], [5, 6], [6, 7], [7, 8], [8, 9], [9, 10],
        [10, 11], [11, 0], [0, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 7], [6, 8], [7, 9], [8,
    ]
    return G

def generate_petersen():
    G = [
        [0, 1], [0, 4], [0, 5], [1, 2], [1, 6], [2, 3], [2, 7], [3, 4], [3, 8], [4, 9],
        [5, 6], [5, 9], [6, 7], [7, 8], [8, 9]
    ]
    return G

def generate_random_3_regular(n):
    while True:
        G = [[] for _ in range(n)]
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([0, 1]) == 1 and (i, j) not in edges and (j, i) not in edges:
                    G[i].append(j)
                    G[j].append(i)
                    edges.add((i, j))
        if is_3_regular(G):
            return G

n_values = [4, 6, 8, 10, 12, 14]

```

```

num_charges = 5
alpha = 0.05
instances_tested = 0
total_width = 0
min_ratio = float('inf')

for n in n_values:
    if n == 6:
        G = generate_prism(n)
    elif n == 8:
        G = generate_möbius_kantor()
    elif n == 10:
        G = generate_petersen()
    else:
        G = generate_random_3_regular(n)

    charges = generate_random_odd_charges(n, num_charges)
    for c in charges:
        instances_tested += 1
        h_md_value = h_md(G)
        T = tseitin_formula(G, c)
        width = resolution_width(T)
        ratio = width / (n * h_md_value)
        total_width += ratio
        min_ratio = min(min_ratio, ratio)

mean_width = total_width / instances_tested
support_fraction = instances_tested / len(n_values) / num_charges

if min_ratio >= alpha and mean_width >= 0.1:
    conjecture_holds = True
    counterexample = ""
else:
    conjecture_holds = False
    counterexample = "mapping_undefined"

return {
    "metric_name": "resolution_width",
    "metric_value": mean_width,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [11, 23, 37, 53, 71] if not sys.argv[1:] else [int(s) for s in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_width} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31fe51cd.py", line 189, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31fe51cd.py", line 159, in run_trial
    h_md_value = h_md(G)
                  ^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_31fe51cd.py", line 52, in h_md
    u, v = e
    ^^^

```

ValueError: too many values to unpack (expected 2)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a ValueError in the h_md function (edge unpacking failure) before producing any data, so no instances were evaluated against the pre-registered criterion.
| next: Fix the edge iteration in h_md (handle multi-edges/self-loops or unpack edges correctly) and rerun the experiment over the specified 3-regular graphs and odd charges.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	104305
2	preregistration	claude_max	opus	0	0	5622
3	novelty	claude_max	opus	0	0	4179
4	novelty	claude_max	opus	0	0	9466
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13021

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21102
7	judge	claude_max	opus	0	0	5325

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 163020 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/c5c4b106a0cf.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c5c4b106a0cf.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c5c4b106a0cf.tar.gz (if generated)